

DATA STRUCTURES & ALGORITHMS · C++

Linked Lists in C++ From Pointers to Placement-Ready

A complete study guide for on-campus placement prep.
Written for someone starting at zero with pointers.

35

SECTIONS

12

PATTERNS

44

CODE BLOCKS

33

PROBLEMS

Prepared for **Hari** · on-campus placement preparation

Companion to **Linked-Lists-Solved-Questions.pdf** · all code verified with `g++ -std=c++17 -Wall`

Generated 06 September 2026

Contents

PART 0 — Pointers, from absolute zero

- 0.1 Memory is a street of houses
- 0.2 A pointer is a variable that stores a house number
- 0.3 nullptr — the pointer that points nowhere
- 0.4 The pointer-vs-value distinction (this is the whole game)
- 0.5 struct — bundling data together
- 0.6 Pointer to a struct, and the arrow ->
- 0.7 The heap: making objects that outlive the function
- 0.8 Putting it together: the Node
- 0.9 Building one by hand
- 0.10 The LeetCode node

PART 1 — The linked list itself

- 1.1 Array vs Linked List
- 1.2 The three types
- 1.3 The traversal loop — memorize this shape
- 1.4 The two loop conditions and what they mean

PART 2 — The three techniques that solve 90% of problems

- Technique 1 — The dummy node (a.k.a. sentinel)
- Technique 2 — Two pointers (slow & fast)
- Technique 3 — Reversal (the prev/curr/next dance)

PART 3 — How many variants are there? (The complete taxonomy)

- Pattern 1 — Traversal & basic operations
- Pattern 2 — Fast & slow pointers
- Pattern 3 — Reversal
- Pattern 4 — Deletion
- Pattern 5 — Merge & sort
- Pattern 6 — Cycle detection (Floyd's tortoise and hare)
- Pattern 7 — Intersection of two lists
- Pattern 8 — Rearrangement
- Pattern 9 — Arithmetic on lists

Pattern 10 — Copy / hashing

Pattern 11 — Recursion on lists

Pattern 12 — Design problems

PART 4 — How to approach ANY linked list problem

Step 1 — Draw the list. Always.

Step 2 — Ask: what do I need to hold on to?

Step 3 — Ask the four screening questions

Step 4 — Decide: extra space allowed?

Step 5 — Write the code with the pointer discipline

Step 6 — Dry-run the edge cases before you submit

PART 5 — The bug catalogue

PART 6 — Complexity cheat sheet

PART 7 — The problem list

PART 8 — A four-week plan

Quick reference card

PART 0 — Pointers, from absolute zero

You cannot understand linked lists without understanding pointers. Ten minutes here saves ten hours later.

0.1 Memory is a street of houses

Think of your computer's memory as a long street. Every house has a **number** (an address) and something **inside** it (a value).

```
int x = 42;
```

This does two things:

- Reserves a house on the street for `x`
- Puts `42` inside it

The house number might be something like `0x7ffd4a`. You never chose it — the computer did.

```
address: 0x7ffd4a
+-----+
x --> | 42 |
+-----+
```

0.2 A pointer is a variable that stores a house number

A normal variable holds a value. A **pointer** holds an *address*.

```
int x = 42;
int* p = &x; // p stores the ADDRESS of x
```

Two new symbols:

SYMBOL	NAME	MEANING
<code>&x</code>	address-of	"give me the house number of x"
<code>*p</code>	dereference	"go to the house p points at, and look inside"

```
int x = 42;
int* p = &x;

cout << x;      // 42      -> the value
cout << p;      // 0x7ffd4a -> the address
cout << *p;     // 42      -> follow the pointer, read the value

*p = 99;        // follow the pointer, WRITE a new value
cout << x;      // 99      -> x changed! p was pointing at x
```

Read `int* p` as: "**p is a pointer to an int.**"

0.3 nullptr — the pointer that points nowhere

```
int* p = nullptr;    // p deliberately points at nothing
```

`nullptr` means "this pointer is empty." It is the **end marker** of every linked list.

The single most common crash in linked list code:

```
int* p = nullptr;
cout << *p;    // CRASH — segmentation fault
```

Dereferencing `nullptr` is asking "what's inside house number zero?" There is no house zero. Your program dies.

Rule you will use in every single problem: before you write `p->something`, you must be certain `p != nullptr`.

0.4 The pointer-vs-value distinction (this is the whole game)

There are two completely different operations, and mixing them up is the #1 source of linked list bugs.

```
p = q;    // MOVE the pointer. Now p points where q points.
           // Nothing in memory changed. Only p changed.

*p = *q;   // CHANGE the contents of the house p points at.
           // p still points at the same house. The house's contents changed.
```

In linked list terms:

```
curr = curr->next;    // I walk forward. The list is untouched.
curr->next = something; // I rewire the list. I did not move.
```

Every linked list algorithm is a careful sequence of *walking* and *rewiring*. Keep them separate in your head.

0.5 struct — bundling data together

A `struct` glues several values into one unit.

```
struct Point {
    int x;
    int y;
};

Point a;
a.x = 3;
a.y = 5;
```

Access members with a **dot** when you have the object itself.

0.6 Pointer to a struct, and the arrow ->

```
Point a;  
Point* p = &a;  
  
(*p).x = 3;    // go to the struct, then take member x - ugly  
p->x = 3;      // exactly the same thing - nice
```

`p->x` is pure shorthand for `(*p).x`. In linked lists you will type `->` thousands of times.

0.7 The heap: making objects that outlive the function

```
int* p = new int(42);    // ask the OS for memory, put 42 in it  
delete p;                // give it back
```

Why does this matter? Local variables die when a function returns. Nodes of a linked list must survive. So nodes are **always** created on the heap with `new`.

```
Point* p = new Point();  // p points to a Point that lives until you delete it  
p->x = 3;
```

In LeetCode you rarely call `delete` — the judge cleans up. But in an interview, mentioning that you'd free removed nodes earns you a point.

0.8 Putting it together: the Node

Now the leap. What if a struct contains a pointer **to another struct of its own type**?

```
struct Node {  
    int val;        // the data  
    Node* next;     // the address of the NEXT node  
};
```

That's it. That's a linked list. One node holds a value and the address of the next node. The last node's `next` is `nullptr`.

```
head  
|  
v  
+-----+ +-----+ +-----+  
| 10 | o-----> | 20 | o-----> | 30 | null |  
+-----+ +-----+ +-----+
```

`head` is just a pointer to the first node. **If you lose `head`, you lose the entire list** — there is no way back. Nobody stores a "previous" pointer in a singly linked list.

0.9 Building one by hand

```
Node* head = new Node();
head->val = 10;

head->next = new Node();
head->next->val = 20;

head->next->next = new Node();
head->next->next->val = 30;
head->next->next->next = nullptr;
```

Cleaner, with a constructor:

```
struct Node {
    int val;
    Node* next;
    Node(int v) : val(v), next(nullptr) {}
};

Node* head = new Node(10);
head->next = new Node(20);
head->next->next = new Node(30);
```

0.10 The LeetCode node

Every LeetCode linked list problem gives you exactly this, already defined:

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};
```

Do not redefine it. Just use `ListNode`.

PART 1 — The linked list itself

1.1 Array vs Linked List

	ARRAY / VECTOR	LINKED LIST
Memory	one contiguous block	scattered nodes joined by pointers
Access <code>i</code> -th element	$O(1)$ — jump straight there	$O(n)$ — must walk from head
Insert/delete at front	$O(n)$ — shift everything	$O(1)$ — rewire two pointers
Insert/delete in middle	$O(n)$	$O(1)$ if you already hold the node before it
Extra memory	none	one pointer per node
Cache friendliness	excellent	poor

The one-line summary interviewers want: arrays win at *random access*, linked lists win at *insertion and deletion*.

1.2 The three types

Singly linked — each node points forward only. 95% of interview problems.

```
A -> B -> C -> null
```

Doubly linked — each node points forward *and* backward. Used in LRU Cache, browser history, and `std::list`.

```
null <- A <-> B <-> C -> null
```

```
struct DNode {
    int val;
    DNode* prev;
    DNode* next;
};
```

Circular — the last node points back to the first instead of `nullptr`. Used for round-robin scheduling, Josephus problem.

```
A -> B -> C --+
^               |
+-----+-----+
```


1.3 The traversal loop — memorize this shape

```
ListNode* curr = head;
while (curr != nullptr) {
    // do something with curr->val
    curr = curr->next;
}
```

Note: we walk with a **copy** of `head` named `curr`. Never move `head` itself unless you intend to lose the list.

Counting length:

```
int length(ListNode* head) {
    int n = 0;
    for (ListNode* curr = head; curr; curr = curr->next) n++;
    return n;
}
```

`while (curr)` and `while (curr != nullptr)` are identical. Same for `if (curr)`.

1.4 The two loop conditions and what they mean

```
while (curr)           // stop when curr falls off the end    -> curr ends as nullptr
while (curr->next)      // stop ON the last node               -> curr ends as the tail
```

Use the second whenever you need the **last node** or need to look one step ahead. But it crashes if `head` is `nullptr`, so guard it.

PART 2 — The three techniques that solve 90% of problems

Before the pattern list, learn these three tools. Almost every linked list solution is a combination of them.

Technique 1 — The dummy node (a.k.a. sentinel)

The problem it solves: the head is a special case. Deleting the head, inserting before the head, merging when the first node comes from list B — each needs its own `if`. That's where bugs live.

The fix: create a fake node in front of the real head. Now every real node has a predecessor, so there are no special cases.

```
ListNode dummy(0);           // a real object on the stack
dummy.next = head;
ListNode* prev = &dummy;

// ... do the work, using prev ...

return dummy.next;           // the (possibly new) head
```

Or with `new`:

```
ListNode* dummy = new ListNode(0, head);
```

Rule of thumb: if the head node might be removed or replaced, use a dummy node. No exceptions. This one habit eliminates a whole class of bugs.

Example — remove all nodes with value `val`:

Without dummy (painful):

```
while (head && head->val == val) head = head->next; // special case
if (!head) return nullptr;
ListNode* curr = head;
while (curr->next) { ... }
```

With dummy (clean):

```
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* curr = &dummy;
    while (curr->next) {
        if (curr->next->val == val) curr->next = curr->next->next;
        else curr = curr->next;
    }
    return dummy.next;
}
```

Notice the `if/else`: when you delete, you do **not** advance, because a new node just slid into that position.

Technique 2 — Two pointers (slow & fast)

Two pointers moving at different speeds, or starting at different offsets. This single idea solves the middle, cycles, the k-th from the end, and palindromes.

Speed variant — slow moves 1, fast moves 2. When fast reaches the end, slow is at the middle.

```
ListNode* slow = head;
ListNode* fast = head;
while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}
// slow is the middle
```

The condition `fast && fast->next` is critical:

- `fast` guards against fast having already fallen off
- `fast->next` guards `fast->next->next` from crashing
- Order matters — C++ short-circuits `&&` left to right

For even-length lists this lands on the **second** middle (1,2,3,4 → 3). To get the **first** middle, start `fast = head->next`.

Gap variant — one pointer starts `k` steps ahead. When the leader hits the end, the follower is `k` from the end.

```
ListNode* fast = head;
for (int i = 0; i < k; i++) fast = fast->next; // build the gap
ListNode* slow = head;
while (fast) { slow = slow->next; fast = fast->next; }
// slow is the k-th node from the end
```

Technique 3 — Reversal (the prev/curr/next dance)

Memorize this. You will write it dozens of times.

```
ListNode* reverse(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nxt = curr->next; // 1. SAVE the rest of the list
        curr->next = prev;          // 2. FLIP this node's arrow backwards
        prev = curr;                // 3. prev moves up
        curr = nxt;                 // 4. curr moves up
    }
    return prev;                    // prev is the new head
}
```

Trace `1 -> 2 -> 3` on paper, drawing the arrows after each iteration. Do it once by hand and you will never forget it.

```
start:  prev=null  curr=1    1 -> 2 -> 3 -> null
iter 1: prev=1     curr=2    null <- 1    2 -> 3 -> null
iter 2: prev=2     curr=3    null <- 1 <- 2    3 -> null
iter 3: prev=3     curr=null  null <- 1 <- 2 <- 3
return prev = 3
```

Why line 1 exists: the moment you write `curr->next = prev`, the link to the rest of the list is destroyed. You must save it first. That is the whole trick.

Golden rule: never overwrite a pointer before you've saved what it was pointing to.

PART 3 — How many variants are there? (The complete taxonomy)

Here is the honest answer: there are **12 patterns**. Every linked list question in an on-campus placement is one of these, or two of them stacked together.

Learn to *recognize* the pattern from the question's wording — that is the actual skill. The code is the easy part.

#	PATTERN	RECOGNITION SIGNAL (WHAT THE QUESTION SAYS)	CORE TOOL
1	Traversal & basic ops	"print", "count", "find", "insert", "delete at position"	the walk loop
2	Fast & slow pointers	"middle", "cycle", "loop", "n-th from end", "palindrome"	two pointers
3	Reversal	"reverse", "backwards", "in groups of k"	prev/curr/next
4	Deletion	"remove", "delete", "duplicates"	dummy node
5	Merge & sort	"two sorted lists", "k lists", "sort the list"	dummy + merge
6	Cycle detection	"does it loop", "where does the loop start"	Floyd's algorithm
7	Intersection & alignment	"where do two lists meet", "common node"	length diff / switching
8	Rearrangement	"reorder", "odd-even", "partition", "rotate", "swap pairs"	split + reverse + merge
9	Arithmetic on lists	"add two numbers", "digits", "plus one", "multiply"	carry propagation
10	Copy / hashing	"random pointer", "deep copy", "clone"	hash map or interleaving
11	Recursion	short lists, "solve recursively", tree-ish structure	recursion + call stack
12	Design	"implement LRU cache", "design a linked list", "flatten multilevel"	doubly linked + map

Below: what each one is, when it shows up, and the template.

Pattern 1 — Traversal & basic operations

You'll see: "insert a node at position k", "find the length", "search for a value", "get the nth node", "print in reverse".

Idea: walk the list with a `curr` pointer. To modify anything, you need to be standing on the node *before* the target.

```

// Length
int length(ListNode* head) {
    int n = 0;
    for (ListNode* c = head; c; c = c->next) n++;
    return n;
}

// Search
bool contains(ListNode* head, int target) {
    for (ListNode* c = head; c; c = c->next)
        if (c->val == target) return true;
    return false;
}

// Insert at position (0-indexed), returns new head
ListNode* insertAt(ListNode* head, int pos, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    for (int i = 0; i < pos && prev->next; i++) prev = prev->next;
    ListNode* node = new ListNode(val);
    node->next = prev->next;    // point new node forward FIRST
    prev->next = node;        // then link it in
    return dummy.next;
}

// Get last node
ListNode* getTail(ListNode* head) {
    if (!head) return nullptr;
    while (head->next) head = head->next;
    return head;
}

```

The insertion order matters. `node->next = prev->next` before `prev->next = node`. Reverse the order and you point the node at itself.

Traps: empty list; position beyond the end; position 0 (this is exactly why the dummy exists).

Pattern 2 — Fast & slow pointers

You'll see: "middle of the list", "n-th node from the end", "is it a palindrome", "delete the middle".

Idea: two pointers, different speeds or a fixed head start. Solves in **one pass with O(1) memory** what would otherwise take two passes.

```

// Middle (second middle for even length)
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// First middle for even length (needed when you SPLIT the list)
ListNode* firstMiddle(ListNode* head) {
    ListNode *slow = head, *fast = head->next;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

// Remove n-th node from the end
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode *slow = &dummy, *fast = &dummy;
    for (int i = 0; i < n; i++) fast = fast->next; // build gap of n
    while (fast->next) { slow = slow->next; fast = fast->next; }
    slow->next = slow->next->next; // slow sits BEFORE the target
    return dummy.next;
}

```

```

// Palindrome check in O(1) space - this is patterns 2 + 3 combined
bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;

    ListNode *slow = head, *fast = head->next; // first middle
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }

    ListNode* second = reverseList(slow->next); // reverse the back half
    slow->next = nullptr;

    ListNode *p = head, *q = second;
    bool ok = true;
    while (p && q) {
        if (p->val != q->val) { ok = false; break; }
        p = p->next; q = q->next;
    }
    slow->next = reverseList(second); // restore the list (polite)
    return ok;
}

```

Why the dummy in `removeNthFromEnd`? Because `n` might equal the list length, meaning you delete the head. Without the dummy that's a special case.

Traps: the `fast && fast->next` order; even vs odd length; forgetting that you need the node *before* the one you delete.

Pattern 3 — Reversal

You'll see: "reverse the list", "reverse between positions `m` and `n`", "reverse in groups of `k`", "reverse alternate `k` nodes".

Idea: flip every arrow. Iterative is $O(1)$ space; recursive is $O(n)$ stack.

```
// Full reverse – iterative (preferred)
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nxt = head->next;
        head->next = prev;
        prev = head;
        head = nxt;
    }
    return prev;
}

// Full reverse – recursive
ListNode* reverseRec(ListNode* head) {
    if (!head || !head->next) return head; // base: 0 or 1 node
    ListNode* newHead = reverseRec(head->next);
    head->next->next = head; // the node after me points back at me
    head->next = nullptr; // I become the new tail
    return newHead;
}

// Reverse a sublist [left, right], 1-indexed
ListNode* reverseBetween(ListNode* head, int left, int right) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    for (int i = 1; i < left; i++) prev = prev->next; // node before the segment

    ListNode* curr = prev->next;
    // head-insertion: repeatedly pull the node after curr to the front of the segment
    for (int i = 0; i < right - left; i++) {
        ListNode* nxt = curr->next;
        curr->next = nxt->next;
        nxt->next = prev->next;
        prev->next = nxt;
    }
    return dummy.next;
}

// Reverse in groups of k (leftover tail stays as-is)
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* node = head;
    for (int i = 0; i < k; i++) { // are there k nodes left?
        if (!node) return head; // no -> leave the remainder alone
        node = node->next;
    }
    // node now points just past this group
    ListNode* prev = reverseKGroup(node, k); // solve the rest first
    ListNode* curr = head;
    for (int i = 0; i < k; i++) { // reverse this group onto prev
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

Traps: losing `next` before flipping; 1-indexed vs 0-indexed in `reverseBetween`; in k-group, whether the leftover tail should be reversed (read the problem — usually not).

Pattern 4 — Deletion

You'll see: "remove all nodes with value x", "remove duplicates", "delete the node given only that node".

Idea: to delete a node, you rewire its **predecessor**. Always use a dummy so the head is not special.

```
// Remove all nodes equal to val
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* curr = &dummy;
    while (curr->next) {
        if (curr->next->val == val) curr->next = curr->next->next;
        else curr = curr->next;
    }
    return dummy.next;
}

// Sorted list -> keep ONE copy of each value
ListNode* deleteDuplicates(ListNode* head) {
    ListNode* curr = head;
    while (curr && curr->next) {
        if (curr->val == curr->next->val) curr->next = curr->next->next;
        else curr = curr->next;
    }
    return head;
}

// Sorted list -> delete EVERY value that appears more than once
ListNode* deleteAllDuplicates(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    ListNode* curr = head;
    while (curr) {
        if (curr->next && curr->val == curr->next->val) {
            int dup = curr->val;
            while (curr && curr->val == dup) curr = curr->next; // skip the whole run
            prev->next = curr; // prev does NOT move
        } else {
            prev = curr;
            curr = curr->next;
        }
    }
    return dummy.next;
}

// Delete a node when you're given ONLY that node (never the tail)
void deleteNode(ListNode* node) {
    node->val = node->next->val; // steal the next node's value
    node->next = node->next->next; // then unlink the next node
}
```

Traps: advancing the pointer after a deletion (don't — a new node slid in); the difference between "keep one copy" and "remove all copies".

Pattern 5 — Merge & sort

You'll see: "merge two sorted lists", "merge k sorted lists", "sort a linked list", "insertion sort a list".

Idea: build the answer with a dummy and a **tail** pointer, picking the smaller head each time. Merge sort is the natural sort for linked lists because splitting is free and no random access is needed.

```

// Merge two sorted lists
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;    // attach whatever remains
    return dummy.next;
}

// Merge sort on a list – O(n log n) time, O(log n) stack
ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;

    // split at the FIRST middle so both halves are non-empty
    ListNode *slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = slow->next;
    slow->next = nullptr;    // cut

    return mergeTwoLists(sortList(head), sortList(second));
}

// Merge k sorted lists – pairwise merging, O(N log k)
ListNode* mergeKLists(vector<ListNode*>& lists) {
    if (lists.empty()) return nullptr;
    int n = lists.size();
    while (n > 1) {
        int half = (n + 1) / 2;
        for (int i = 0; i < n / 2; i++)
            lists[i] = mergeTwoLists(lists[i], lists[i + half]);
        n = half;
    }
    return lists[0];
}

```

Traps: in `sortList`, using `fast = head` instead of `head->next` gives an empty first half for a 2-node list → infinite recursion. And you *must* cut with `slow->next = nullptr`.

Pattern 6 — Cycle detection (Floyd's tortoise and hare)

You'll see: "does the list have a cycle", "find the node where the cycle begins", "length of the loop", "happy number" (same maths).

Idea: slow moves 1, fast moves 2. If there's a loop, fast laps slow and they collide. If not, fast falls off the end.

```

bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

// Find the first node of the cycle
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { // met inside the loop
            ListNode* p = head;
            while (p != slow) { // move both 1 step until they meet
                p = p->next;
                slow = slow->next;
            }
            return p; // that meeting point is the loop start
        }
    }
    return nullptr;
}

// Length of the cycle
int cycleLength(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            int len = 1;
            for (ListNode* p = slow->next; p != slow; p = p->next) len++;
            return len;
        }
    }
    return 0;
}

```

Why the second phase works (say this in the interview): let L = distance from head to the loop start, C = loop length, and let them meet x steps into the loop. Fast has travelled twice as far as slow, so $2(L + x) = L + x + kC$, which gives $L = kC - x$. That means: walking L steps from the head, and walking L steps forward from the meeting point, both land on the loop start. Hence one pointer from `head`, one from the meeting point, one step each, and they meet at the entrance.

Traps: comparing `slow->val == fast->val` instead of `slow == fast` — compare **addresses**, not values.

Pattern 7 — Intersection of two lists

You'll see: "find the node where two linked lists merge", "common node".

Idea: the lists have different lengths before the junction. Cancel the difference, then walk together.

```

// Trick version: switch heads when you hit the end.
// Both pointers travel exactly lenA + lenB steps, so they sync up.
ListNode* getIntersectionNode(ListNode* a, ListNode* b) {
    if (!a || !b) return nullptr;
    ListNode *p = a, *q = b;
    while (p != q) {
        p = p ? p->next : b;    // when p ends, restart on b
        q = q ? q->next : a;    // when q ends, restart on a
    }
    return p;                  // the junction, or nullptr if they never meet
}

// Explicit version: align by length difference
ListNode* getIntersectionByLength(ListNode* a, ListNode* b) {
    int la = length(a), lb = length(b);
    while (la > lb) { a = a->next; la--; }
    while (lb > la) { b = b->next; lb--; }
    while (a != b) { a = a->next; b = b->next; }
    return a;
}

```

Why the trick terminates when there's no intersection: both pointers become `nullptr` at the same time after `lenA + lenB` steps, and `nullptr == nullptr`, so the loop exits.

Traps: compare node *addresses*, not values.

Pattern 8 — Rearrangement

You'll see: "reorder the list", "odd-even list", "partition around x", "rotate right by k", "swap nodes in pairs", "reverse alternate nodes".

Idea: this is the composite pattern — almost always **split → transform → merge**. Recognising that decomposition is the whole battle.

```

// Reorder: L0 -> Ln -> L1 -> Ln-1 -> ...
// = find middle, reverse second half, weave the two halves
void reorderList(ListNode* head) {
    if (!head || !head->next) return;

    ListNode *slow = head, *fast = head->next;           // first middle
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = slow->next;
    slow->next = nullptr;

    ListNode* prev = nullptr;                             // reverse second half
    while (second) {
        ListNode* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }

    ListNode* first = head;                               // weave
    while (prev) {
        ListNode* n1 = first->next;
        ListNode* n2 = prev->next;
        first->next = prev;
        prev->next = n1;
        first = n1;
        prev = n2;
    }
}

// Odd-even: group nodes at odd positions, then nodes at even positions
ListNode* oddEvenList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* odd = head;
    ListNode* even = head->next;
    ListNode* evenHead = even;
    while (even && even->next) {
        odd->next = even->next; odd = odd->next;
        even->next = odd->next; even = even->next;
    }
    odd->next = evenHead;
    return head;
}

// Partition: all nodes < x before all nodes >= x, order preserved
ListNode* partition(ListNode* head, int x) {
    ListNode lessDummy(0), geDummy(0);
    ListNode *less = &lessDummy, *ge = &geDummy;
    while (head) {
        if (head->val < x) { less->next = head; less = less->next; }
        else { ge->next = head; ge = ge->next; }
        head = head->next;
    }
    ge->next = nullptr; // ESSENTIAL: cut the old tail
    less->next = geDummy.next;
    return lessDummy.next;
}

// Rotate right by k
ListNode* rotateRight(ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;
    int n = 1;
    ListNode* tail = head;
    while (tail->next) { tail = tail->next; n++; }
    k %= n; // k can exceed n
    if (k == 0) return head;
    tail->next = head; // make it circular
    ListNode* newTail = head;
    for (int i = 0; i < n - k - 1; i++) newTail = newTail->next;
}

```

```

ListNode* newHead = newTail->next;
newTail->next = nullptr;           // break the circle
return newHead;
}

// Swap every two adjacent nodes
ListNode* swapPairs(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    while (prev->next && prev->next->next) {
        ListNode* a = prev->next;
        ListNode* b = a->next;
        a->next = b->next;
        b->next = a;
        prev->next = b;
        prev = a;
    }
    return dummy.next;
}

```

Traps: forgetting to cut a tail (`ge->next = nullptr` in partition) creates a cycle and hangs the judge; `k % n` in rotate; the two-dummy trick is worth remembering — it appears constantly.

Pattern 9 — Arithmetic on lists

You'll see: "add two numbers represented as linked lists", "add one to the number", "multiply two lists".

Idea: digit-by-digit with a carry, exactly like primary-school addition. If digits are stored **reversed** (least significant first) it's a simple walk. If stored **forward**, you either reverse both lists first or use two stacks.

```

// Digits stored in REVERSE order: 2->4->3 means 342
ListNode* addTwoNumbers(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    int carry = 0;
    while (a || b || carry) {
        int sum = carry;
        if (a) { sum += a->val; a = a->next; }
        if (b) { sum += b->val; b = b->next; }
        carry = sum / 10;
        tail->next = new ListNode(sum % 10);
        tail = tail->next;
    }
    return dummy.next;
}

// Digits stored in FORWARD order: 7->2->4->3 means 7243. Use stacks.
ListNode* addTwoNumbersForward(ListNode* a, ListNode* b) {
    stack<int> sa, sb;
    for (; a; a = a->next) sa.push(a->val);
    for (; b; b = b->next) sb.push(b->val);
    ListNode* head = nullptr;
    int carry = 0;
    while (!sa.empty() || !sb.empty() || carry) {
        int sum = carry;
        if (!sa.empty()) { sum += sa.top(); sa.pop(); }
        if (!sb.empty()) { sum += sb.top(); sb.pop(); }
        carry = sum / 10;
        head = new ListNode(sum % 10, head); // build by prepending
    }
    return head;
}

```

Traps: the trailing carry — `while (a || b || carry)` handles `999 + 1` in one line. Forgetting `|| carry` is the classic bug.

Pattern 10 — Copy / hashing

You'll see: "copy a list with random pointers", "clone the list", "deep copy".

Idea: you need a mapping from old node → new node. Either a hash map ($O(n)$ space) or the interleaving trick ($O(1)$ space).

```

struct RNode {
    int val;
    RNode* next;
    RNode* random;
    RNode(int v) : val(v), next(nullptr), random(nullptr) {}
};

// Approach A – hash map, easy to explain
RNode* copyWithMap(RNode* head) {
    unordered_map<RNode*, RNode*> mp;
    for (RNode* c = head; c; c = c->next) mp[c] = new RNode(c->val);
    for (RNode* c = head; c; c = c->next) {
        mp[c]->next = c->next ? mp[c->next] : nullptr;
        mp[c]->random = c->random ? mp[c->random] : nullptr;
    }
    return head ? mp[head] : nullptr;
}

// Approach B – O(1) extra space, three passes
RNode* copyInterleave(RNode* head) {
    if (!head) return nullptr;
    // 1. weave copies in: A -> A' -> B -> B' -> ...
    for (RNode* c = head; c; c = c->next->next) {
        RNode* copy = new RNode(c->val);
        copy->next = c->next;
        c->next = copy;
    }
    // 2. set random pointers: the copy of X is X->next
    for (RNode* c = head; c; c = c->next->next)
        if (c->random) c->next->random = c->random->next;
    // 3. unweave the two lists
    RNode* newHead = head->next;
    for (RNode* c = head; c; c = c->next) {
        RNode* copy = c->next;
        c->next = copy->next;
        if (copy->next) copy->next = copy->next->next;
    }
    return newHead;
}

```

Traps: setting the random pointer to the *original* node instead of its copy; forgetting to restore the original list in approach B.

Pattern 11 — Recursion on lists

You'll see: any list problem, really — but especially "reverse", "merge", "palindrome", "flatten", "k-group".

Idea: a linked list *is* a recursive structure: a node plus a smaller linked list. The template is always the same.

```

if (base case) return something;
result = solve(head->next);    // trust the recursion for the rest
combine head with result;
return;

```



```

// Merge two sorted lists, recursively
ListNode* mergeRec(ListNode* a, ListNode* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->val <= b->val) { a->next = mergeRec(a->next, b); return a; }
    else { b->next = mergeRec(a, b->next); return b; }
}

// Remove elements, recursively
ListNode* removeRec(ListNode* head, int val) {
    if (!head) return nullptr;
    head->next = removeRec(head->next, val);
    return head->val == val ? head->next : head;
}

```

Trap: recursion uses $O(n)$ stack. With n up to 10^5 (common on LeetCode), deep recursion can stack-overflow. Say this out loud in an interview and offer the iterative version — it's an easy signal of maturity.

Pattern 12 — Design problems

You'll see: "design a linked list", "implement an LRU cache", "flatten a multilevel doubly linked list", "design a browser history".

Idea: these combine a doubly linked list (for $O(1)$ removal from the middle) with a hash map (for $O(1)$ lookup). **LRU Cache is the single most-asked design question in placements.**

```

class LRUCache {
    struct Node {
        int key, val;
        Node *prev, *next;
        Node(int k, int v) : key(k), val(v), prev(nullptr), next(nullptr) {}
    };
    int cap;
    unordered_map<int, Node*> mp;
    Node *head, *tail;                                // head = most recent, tail = least

    void remove(Node* n) {
        n->prev->next = n->next;
        n->next->prev = n->prev;
    }
    void insertFront(Node* n) {
        n->next = head->next;
        n->prev = head;
        head->next->prev = n;
        head->next = n;
    }
public:
    LRUCache(int capacity) : cap(capacity) {
        head = new Node(0, 0);                        // sentinels at both ends
        tail = new Node(0, 0);                        // so remove/insert need no null checks
        head->next = tail;
        tail->prev = head;
    }
    int get(int key) {
        if (!mp.count(key)) return -1;
        Node* n = mp[key];
        remove(n);
        insertFront(n);                               // touching it makes it most recent
        return n->val;
    }
    void put(int key, int value) {
        if (mp.count(key)) {
            Node* n = mp[key];
            n->val = value;
            remove(n);
            insertFront(n);
            return;
        }
        if ((int)mp.size() == cap) {                 // evict the least recent
            Node* lru = tail->prev;
            remove(lru);
            mp.erase(lru->key);
            delete lru;
        }
        Node* n = new Node(key, value);
        mp[key] = n;
        insertFront(n);
    }
};

```

Why sentinels at both ends: with a dummy head and a dummy tail, `remove()` never has to check for `nullptr`. Four lines, no branches. This is the dummy-node idea taken to its conclusion.

PART 4 — How to approach ANY linked list problem

This is the part most people skip, and it is the part that actually wins interviews. Follow these six steps every single time until it becomes automatic.

Step 1 — Draw the list. Always.

Never solve a linked list problem in your head. Take paper. Draw boxes and arrows.

```
head
  v
[1] -> [2] -> [3] -> [4] -> [5] -> null
```

Then draw what the answer should look like. The gap between the two pictures *is* the algorithm. Most "hard" linked list problems become obvious the moment you draw the before and after.

Step 2 — Ask: what do I need to hold on to?

Every rewiring needs a set of pointers. Before writing code, name them:

- Deleting a node → I need the node **before** it
- Reversing → I need `prev`, `curr`, `next`
- Splitting → I need the node before the cut, and the head of the second half
- Merging → I need `tail` of the result plus a head for each input

Write the pointer names down first. Then the code writes itself.

Step 3 — Ask the four screening questions

These map the problem to a pattern in about ten seconds:

1. **Can the head change or be deleted?** → use a **dummy node**.
2. **Do I need a position relative to the end, or the middle?** → **two pointers**.
3. **Does the output order run backwards, or in chunks?** → **reversal**.
4. **Do I need to look up a node by identity?** → **hash map**.

Most problems answer "yes" to one or two of these. That combination *is* your solution.

Step 4 — Decide: extra space allowed?

Ask the interviewer. It changes everything.

	O(N) SPACE ALLOWED	O(1) SPACE REQUIRED
Palindrome	copy to a vector, check two-pointer	reverse the second half
Find middle	count, then walk $n/2$	fast & slow
Detect cycle	hash set of visited nodes	Floyd's algorithm
Copy random list	hash map old→new	interleave the copies

Interview strategy: state the $O(n)$ -space solution first — it shows you can solve the problem — then say "and I can do this in $O(1)$ space with..." That progression is exactly what interviewers are grading.

Step 5 — Write the code with the pointer discipline

Three rules, in order of how often they're violated:

- 1. Save before you overwrite.** `next = curr->next;` *before* `curr->next = prev;`
- 2. Check before you dereference.** Never write `p->next->next` without knowing `p` and `p->next` are non-null.
- 3. Cut every tail you create.** If you split a list, the first half's last node must be set to `nullptr`. If you build a new list from old nodes, terminate it. A forgotten cut makes a cycle, and the judge hangs forever.

Step 6 — Dry-run the edge cases before you submit

Run your code *on paper* against this checklist. Every one of these has been the reason someone failed an interview:

CASE	INPUT	WHY IT BREAKS THINGS
Empty list	<code>head == nullptr</code>	any <code>head-></code> crashes immediately
Single node	<code>[1]</code>	<code>head->next</code> is null; fast/slow loops behave oddly
Two nodes	<code>[1,2]</code>	the classic off-by-one; breaks <code>sortList</code> if you split wrong
Operation at the head	delete/reverse from position 1	needs a dummy node
Operation at the tail	last node	<code>curr->next</code> is null
All elements identical	<code>[2,2,2]</code>	breaks deduplication logic
k larger than length	rotate by 100 on 3 nodes	needs <code>k % n</code>
k == length	remove n-th from end	you're deleting the head
Duplicate values	<code>[1,2,1]</code>	reminder to compare pointers, not values

Do this for every problem. It takes ninety seconds and it is the difference between "works" and "works on the judge."

PART 5 — The bug catalogue

The same eight bugs, over and over. Learn them once.

1. Losing the rest of the list

```
curr->next = prev;           // WRONG — the rest of the list is now unreachable
ListNode* nxt = curr->next; // too late

ListNode* nxt = curr->next; // RIGHT — save first
curr->next = prev;
```

2. Null dereference

```
while (fast->next && fast) // WRONG order — checks fast->next on a null fast
while (fast && fast->next) // RIGHT — && short-circuits left to right
```

3. Advancing after a delete

```
if (curr->next->val == val) { curr->next = curr->next->next; curr = curr->next; } // WRONG
// skips the node that slid in
if (curr->next->val == val) curr->next = curr->next->next; // RIGHT — don't move
else curr = curr->next;
```

4. Accidentally creating a cycle — forgetting `slow->next = nullptr` after a split, or `ge->next = nullptr` in partition. Symptom: infinite loop / TLE.

5. Comparing values instead of addresses

```
if (slow->val == fast->val) // WRONG — different nodes can share a value
if (slow == fast)         // RIGHT — cycle/intersection is about identity
```

6. Returning `head` when the head changed — after a reversal, `head` is the *tail*. Return `prev`. After a dummy-node algorithm, return `dummy.next`, never `head`.

7. Forgetting the final carry — `while (a || b)` misses `999 + 1`. Use `while (a || b || carry)`.

8. Off-by-one in 1-indexed problems — `reverseBetween(head, left, right)` is 1-indexed. `for (int i = 1; i < left; i++)` walks to the node *before* the segment. Draw it if unsure.

PART 6 — Complexity cheat sheet

OPERATION	TIME	SPACE
Traverse / length / search	$O(n)$	$O(1)$
Insert / delete at head	$O(1)$	$O(1)$
Insert / delete at tail (no tail pointer)	$O(n)$	$O(1)$
Access k-th element	$O(n)$	$O(1)$
Reverse (iterative)	$O(n)$	$O(1)$
Reverse (recursive)	$O(n)$	$O(n)$ stack
Find middle	$O(n)$	$O(1)$
Detect cycle (Floyd)	$O(n)$	$O(1)$
Merge two sorted	$O(n+m)$	$O(1)$
Merge sort a list	$O(n \log n)$	$O(\log n)$ stack
Merge k lists (pairwise)	$O(N \log k)$	$O(1)$
Copy with random (map)	$O(n)$	$O(n)$
Copy with random (interleave)	$O(n)$	$O(1)$
LRU get / put	$O(1)$	$O(\text{capacity})$

The standard interview answer: " $O(n)$ time, $O(1)$ space" is the target for almost every singly-linked-list problem. If your solution needs $O(n)$ space, there is usually a pointer trick that removes it.

PART 7 — The problem list

Work through these in order. Do not skip to the hard ones — the patterns compound.

Tier 1 — Learn the mechanics (do all of these first)

#	PROBLEM	PATTERN
707	Design Linked List	1 — basic ops
876	Middle of the Linked List	2 — fast/slow
206	Reverse Linked List	3 — reversal
203	Remove Linked List Elements	4 — dummy node
83	Remove Duplicates from Sorted List	4 — deletion
21	Merge Two Sorted Lists	5 — merge
141	Linked List Cycle	6 — Floyd
160	Intersection of Two Linked Lists	7 — alignment
234	Palindrome Linked List	2 + 3 — combo
237	Delete Node in a Linked List	4 — the trick one

Ten problems. When you can write all ten from a blank page without looking, you have the mechanics.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
19	Remove Nth Node From End of List	2 — gap pointers
2	Add Two Numbers	9 — arithmetic
142	Linked List Cycle II	6 — find loop start
92	Reverse Linked List II	3 — sublist reversal
82	Remove Duplicates from Sorted List II	4 — remove all copies
24	Swap Nodes in Pairs	8 — rearrangement
61	Rotate List	8 — rotate
86	Partition List	8 — two dummies
328	Odd Even Linked List	8 — weaving
143	Reorder List	8 — split+reverse+merge
148	Sort List	5 — merge sort
138	Copy List with Random Pointer	10 — hashing
146	LRU Cache	12 — design ★

★ **LRU Cache is asked more than any other linked list question in on-campus drives.**

Amazon, Microsoft, Adobe, Flipkart, Uber — all of them. Be able to write it cold.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
25	Reverse Nodes in k-Group	3 — hard reversal
23	Merge k Sorted Lists	5 — k-way merge
445	Add Two Numbers II	9 — stacks
430	Flatten a Multilevel Doubly Linked List	12 — DFS on lists
147	Insertion Sort List	5 — sorting
1721	Swapping Nodes in a Linked List	2 — gap pointers
109	Convert Sorted List to BST	conversion
725	Split Linked List in Parts	1 + splitting
622	Design Circular Queue	12 — design
460	LFU Cache	12 — design, hard

The classic Indian-placement questions (GfG / interview favourites)

These come up in written rounds and technical interviews specifically:

- Reverse a linked list in groups of size k
- Detect and **remove** a loop from a linked list
- Find the middle in a single pass
- Merge two sorted linked lists **without extra space**
- Delete the middle node
- Check if a linked list is a palindrome in $O(1)$ space
- Add 1 to a number represented as a linked list
- Flatten a linked list with `next` and `bottom` pointers
- Clone a linked list with next and random pointers
- Rotate a linked list by k places
- Segregate even and odd nodes
- Intersection point of two linked lists
- Josephus problem using a circular linked list

PART 8 — A four-week plan

Assumes about 1.5 hours a day.

Week 1 — Foundation. Re-read Part 0 until pointers feel obvious. Implement a linked list from scratch in C++: insert at head, insert at tail, insert at position, delete by value, search, print, reverse. Do **not** use LeetCode this week — build it yourself in a file and run it. Then solve Tier 1, problems 707, 876, 206, 203, 83.

Week 2 — The three techniques. Finish Tier 1. For each of the dummy node, fast/slow, and reversal templates, write it from memory three days in a row. Start Tier 2: 19, 2, 142, 92, 82.

Week 3 — Composite problems. Tier 2 the rest: 24, 61, 86, 328, 143, 148, 138. These are combinations, so consciously name the sub-patterns as you decompose each one. Then write LRU Cache (146) from scratch, twice, on different days.

Week 4 — Hardening. Tier 3. Then re-solve ten Tier 2 problems on a timer, 20 minutes each, on paper, no compiler. This last exercise is the closest thing to the real interview and it will expose exactly which patterns are still shaky.

Ongoing: whenever you fail a problem, don't just read the solution. Identify which of the 12 patterns it was, and which of the 8 bugs you hit. Keep a one-line log. After thirty problems that log tells you precisely what to drill.

Quick reference card

```
// ===== the node =====
struct ListNode { int val; ListNode* next; ListNode(int x):val(x),next(nullptr){} };

// ===== walk =====
for (ListNode* c = head; c; c = c->next) { /* ... */ }

// ===== dummy node (head might change) =====
ListNode dummy(0); dummy.next = head;
ListNode* prev = &dummy;
/* ... work ... */
return dummy.next;

// ===== fast & slow (middle / cycle) =====
ListNode *slow = head, *fast = head;
while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }

// ===== gap pointers (k-th from end) =====
ListNode *slow = &dummy, *fast = &dummy;
for (int i = 0; i < k; i++) fast = fast->next;
while (fast->next) { slow = slow->next; fast = fast->next; }

// ===== reverse =====
ListNode* prev = nullptr;
while (head) { ListNode* nxt = head->next; head->next = prev; prev = head; head = nxt; }
return prev;

// ===== split at first middle =====
ListNode *slow = head, *fast = head->next;
while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
ListNode* second = slow->next;
slow->next = nullptr;           // CUT

// ===== build a new list =====
ListNode dummy(0); ListNode* tail = &dummy;
/* tail->next = node; tail = tail->next; */
return dummy.next;
```

The three rules, one last time:

1. Save `next` before you overwrite it.
2. Check for `nullptr` before you dereference.
3. Cut every tail you create.